

Phase 4 Report

Kubernetes Workload Analyzer — rule engine, evidence, cost, findings API

Phase status	DONE (2026-09-26); KRR comparison deferred
Report date	2026-09-26
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	3807a0e , merged as 11848d6 (PR #9)
Delegation	Phases 4-8 full implementation autonomy granted by the owner 2026-09-26 (AGENTS.md)
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 4 was for
2. Evidence builder: resolving pods without pods access
3. Self-calibrated data coverage (R005)
4. Rules R001-R004
5. Three real bugs found via live verification
6. Cost model
7. Live verification against demo/expected-findings.yaml
8. A genuine non-fixture finding
9. Idempotency and graceful degradation
10. Git and GitHub workflow
11. Current state and operating
12. Open items and next phase

1. What Phase 4 was for

Phase 4 is the analytical core: turning Prometheus data into evidence-based findings. Built: the evidence builder, R001-R004 as pure functions, the R005 data-quality gate, the cost estimator, the findings orchestrator (ranking, stable IDs), and `/api/findings`. Done condition (`PHASES.md`): output matches `demo/expected-findings.yaml` for all 9 fixtures. This is the first phase run under the owner's Phase 4-8 delegation (`AGENTS.md`): full implementation autonomy, still bound by the Correctness/Security/Code rules as a hard floor.

2. Evidence builder: resolving pods without pods access

D-007 grants no `pods` API access, but usage metrics (`container_cpu_usage_seconds_total` etc.) only carry a `pod` label, not an owner reference. Pods are resolved entirely through kube-state-metrics series instead: `kube_pod_owner` joined to `kube_replicaset_owner` on the ReplicaSet name, using `label_replace` on both sides to align the join key (`internal/evidence/builder.go`'s `resolvePodNames`) — verified live to return exactly the right pod name before being relied on elsewhere.

```
label_replace(kube_pod_owner{namespace="demo", owner_kind="ReplicaSet"}, "rsname", "$1", "owner_name",
"(.*)")
  * on(namespace, rsname) group_left()
label_replace(kube_replicaset_owner{namespace="demo", owner_kind="Deployment", owner_name="cpu-over-
requested"}, "rsname", "$1", "replicaset", "(.*)")
```

3. Self-calibrated data coverage (R005)

Coverage is **not** assumed from a fixed scrape interval. An assumed 30s interval was checked against real fixture data and found off by more than 2x (823 samples over ~15h where the assumption predicted ~1816) — likely scrape-job cadence differences plus real gaps. Coverage instead comes from the actual sample timestamps returned: $\text{span covered} \div \text{window}$, penalized for the single largest gap found. Tiers: **insufficient** (below 80% coverage even at the 30m minimum) · **stale** (newest sample older than 5m) · **ok** at 30m/24h/7d, mapping to confidence LOW/MEDIUM/HIGH respectively.

4. Rules R001-R004

ID	Rule	Condition	Gate
R001	CPU over-requested	p95(usage) < 0.5x request AND request-p95 ≥ 100m AND throttled < 5% AND not HPA-CPU-coupled AND not bursty (p99/p50 ≤ 4)	R005 ok
R002	Memory over-requested	max(working set) < 0.5x request AND request-max ≥ 64Mi AND no OOM in 24h	R005 ok
R003	Frequent restarts	restart increase ≥ 3 in 1h OR ≥ 10 in 24h	restart counter present
R004	OOM	last_terminated_reason == OOMKilled AND restarts increased in 24h	termination reason + restart counter present

R003/R004 deliberately skip the R005 coverage-tier gate: a restart counter is meaningful with far less history than a percentile calculation needs, and the 30m/24h/7d bar would under-serve health findings that should fire quickly. **Not implemented** (named so it isn't silently forgotten): R004's original design also included a near-limit warning (`max(working set) ≥ 0.9x limit`), left out because no demo fixture exercises it — candidate for a later phase.

5. Three real bugs found via live verification

Not made to pass a test written after the fact — each was caught by running against the real cluster, then turned into a regression test.

Bug 1 — a `rate()` window that silently defeated the bursty guard. The CPU usage query originally used `rate(...[5m])`. For the `cpu-spike` fixture (30s burst every 300s), a 5-minute lookback smoothed every evaluated point into a near-constant ~10% average, destroying the p50/p99 spread the burstiness guard depends on. Diagnosed with a standalone debug tool showing `P50=0.1056`, `P95=0.1108`, `P99=0.1180` — nearly flat, burst smoothed away entirely. Fixed to `rate(...[2m])`, short enough to leave idle-only points near zero and burst-overlapping points visibly elevated.

Bug 2 — a zero median inverted the burstiness signal. `Burstiness() = P99/P50` returned 0 when P50 was exactly 0, treating "no meaningful median" the same as "no signal at all" — exactly backwards for a workload idle more than half the time with a real usage tail. Fixed to return `+Inf` in that case, now covered by `TestBurstiness_ZeroMedianWithRealTail_IsInfinite`.

Bug 3 — a misleading confidence-reason message. The LOW-tier confidence reason string used the medium-window constant twice instead of the minimum-window constant, so the printed explanation didn't match the actual threshold being applied. Fixed and covered.

Full detail: [docs/requirements.md §3](#), "Implementation notes".

6. Cost model

Allocation = `max(request, observed usage)` (same idea as OpenCost). Cost fields are always labelled **Estimated**; the reduction is a **potential difference**, never "savings" (`internal/cost/cost.go`). No built-in default price: `PricingUnset` fails closed if either `CPU_CORE_HOUR_USD` or `MEMORY_GIB_HOUR_USD` is missing — a finding either has a fully assumption-labelled cost or none at all, never a silent guess.

7. Live verification against demo/expected-findings.yaml

End-to-end against the real cluster with real pricing configured — output matched `demo/expected-findings.yaml` exactly for all 9 fixtures: `right-sized` and `hpa-coupled` and `new-workload` and `pending` produced no R001/R002 (negative controls), `cpu-over-requested`/`memory-over-requested` fired R001/R002 at MEDIUM confidence, `cpu-spike` was correctly suppressed by the bursty guard, `oom` fired R004+R003 at `critical`, `crashloop` fired R003 only (R004 correctly did not fire — reason is `Error`, not `OOMKilled`).

8. A genuine non-fixture finding

The live analyzer found a real, non-fixture R002 finding on `kube-system/metrics-server` — 200Mi requested vs ~59Mi observed. Not planted, not a demo scenario: it confirms the rule engine generalizes to real cluster workloads beyond what it was built and tuned against.

9. Idempotency and graceful degradation

Finding IDs are `sha256(cluster + rule + namespace + workload + container)`, truncated to 16 hex characters — verified idempotent: the same finding IDs across independent runs, so re-running never duplicates a finding. With real pricing wired in, Prometheus was killed and restored live again (as in Phase 3, now with the full findings pipeline attached): findings stayed sticky, `/readyz` 503'd and recovered, `SIGTERM` shutdown stayed clean.

10. Git and GitHub workflow

Delegation recorded first: chore: record Phase 4-8 implementation delegation in AGENTS.md (5303ab5, PR #8, merged 8067f33). Then branch phase-4 → commit 3807a0e ("feat(phase-4): rule engine (R001-R004), evidence, cost, findings API") → PR #9 → merged as 11848d6 → branch deleted.

11. Current state and operating

```
kubectl -n monitoring port-forward svc/kps-kube-prometheus-stack-prometheus 9090:9090
cd backend
CPU_CORE_HOUR_USD=0.0316 MEMORY_GIB_HOUR_USD=0.0042 PRICE_SOURCE="example only" go run ./cmd/analyzer
curl localhost:8080/api/findings
```

12. Open items and next phase

- KRR comparison deferred to a later phase — not blocking, no demo fixture needs it.
- R004 near-limit-memory variant not built (backlog).
- No dashboard yet — `/api/findings` has no UI consumer until Phase 5.